

(Vasavi Urhakolla) 56

Handwritten Reflection Questions

1. Which optimization produced the greatest improvement?

⇒ Transitioning from spawning unbounded raw threads (threading.Thread) to using a thread pool (ThreadPoolExecutor) alongside explicit socket timeouts provided the most significant improvement. It directly stabilized CPU and memory utilization under concurrent client loads, preventing system degradation and thread exhaustion errors during high traffic.

2. Why is scalability important?

↓ Scalability ensures that an application can gracefully handle increasing numbers of concurrent users and higher message volumes without crashing, freezing, or suffering severe performance loss. In network applications like chat systems, scalable architectures ensure resource efficiency (e.g., controlling thread overhead and memory consumption) and maintain consistent latency and responsiveness across all connected clients as demand grows.

Page No.
 Date
 3. what reliability issues did you discover?

⇒ I discover ghost connections remaining open after sudden client disconnects, unhandled socket timeouts causing thread crashes, and race conditions when modifying active user lists without thread locking (LOCK)

4. which optimization was the most difficult

⇒ Implementing thread-safe inactivity timeout handling was the most difficult. It required precisely tracking timestamps after authentication without triggering premature disconnects during non-blocking socket loops

5. what additional improvements are required to support 100 users?

⇒ Supporting 200 users would require ~~user~~ switching to asynchronous non-blocking I/O (using `epoll`), using a dedicated database like PostgreSQL or Redis for session state, adding TLS/SSL encryption and deploying a load balancer.